

ARCHITECTURE

기획부터 배포까지의 AI 활용 구조

01 · 디자인

Stitch AI

디자인 역량이 부족한 부분을 보완하기 위해 Stitch AI로 화면 디자인을 생성하고, 프롬프트로 기획한 부분을 계속 수정했다. Stitch AI는 MCP를 제공해 프론트엔드 에이전트가 Stitch에서 만든 화면 디자인을 직접 읽어올 수 있다는 장점이 있어 채택했다.

02 · 인프라 구성

EC2·S3 (AI 미사용)

인프라는 강점 영역이라 판단해 AI 없이 직접 배포를 진행했다. Terraform으로 EC2·S3를 프로비저닝하고, 01번 프로젝트의 Ansible 기반 Kubernetes 설치로 그 위에 백엔드·프론트엔드를 배포했다. 이미지는 S3에 분리 저장했다.

03 · 백엔드

컨벤션 강제 준수

계층 구조·CQRS·DTO 3계층 등 백엔드 컨벤션을 문서로 미리 정의해 reference로 고정하고, 전담 SubAgent가 이를 따라 코드를 작성하도록 구성해 일관성을 확보했다. 코드 생성 뒤 보안·성능 전담 리뷰어 SubAgent가 자동으로 검수·최적화했다.

04 · 프론트엔드

개발/리뷰 파이프라인 구축

프론트엔드는 감각이 부족해 멧쟁이사자처럼 프론트엔드 운영진의 조언으로 도구(Tailwind 등)와 방식을 정했다. Stitch 디자인을 MCP로 연결해 규칙으로 삼고, 개발(fe-developer)·리뷰어(fe-reviewer) Agent로 버그·규칙 준수·빌드 여부를 확인하는 워크플로우를 설계했다.

05 · 공통

make-workflow 스킬

위 개발/리뷰 팀들을 하나씩 손으로 만드는 대신 make-workflow(Skill)로 찍어냈다. workflow-architect(Agent)가 요청을 team-spec으로 먼저 분해하면, make-agent가 에이전트를, skill-author가 이들을 지휘하는 Skill을 생성하고, workflow-integration-reviewer가 스킬이 부르는 이름과 실제 에이전트·상하류 계약이 맞물리는지 교차 검증했다. "작성 → 검수 → 통과할 때까지 반복" 원칙을 에이전트·스킬을 만드는 층에도 그대로 적용해, 하네스 자체를 일관되게 생성했다.